The traffic encoding (v9 / v10)

How the surrounding vehicles are turned into a fixed-size input vector of 80 numbers

1. What it is

`encoding.py` describes the traffic around the ego as **80 numbers**, computed every control step (0.1 s). It is meant as the **input of a future learned model**; the controller itself does not use it. Three properties drive its design:

- **Lane-free.** The traffic is sub-lane (bikes filter between cars), so nothing refers to lanes. Every vehicle is described relative to the ego's own body and path.
- **Vehicle-type aware.** Gaps are measured body to body, and each encoded vehicle's length and width are part of the input (a 1.8 m bike and a 12 m bus are different).
- **Fixed size, independent of the number and order of vehicles.** The space around the ego is split into 8 sectors; each sector has a fixed slot, filled by its most critical vehicle (or default values if empty). Dynamics enter through relative velocities, inverse time-to-contact and required braking.

Layout: 8 sectors × 9 values = 72, plus 8 global values = 80.

2. Geometry: body-to-body gaps

All positions are body centres, sizes are the vehicles' actual SUMO sizes, x is along the road, y across it. For a neighbour with length L_v and width W_v at offset (dx, dy) from the ego centre:

$$g_x = |dx| - (L_{ego} + L_v) / 2$$ longitudinal box gap (negative: bodies overlap along the road)

$$g_y = |dy| - (W_{ego} + W_v) / 2$$ lateral box gap (negative: bodies overlap across the road)

Using gaps instead of centre distances is what makes the encoding size- and type-aware: the same centre offset means a smaller gap to a bus than to a bike.

3. The 8 sectors

Each neighbour is put in exactly one sector, from two relations:

relation	rule	result
along the road	$g_x \geq 0$ and dx > 0 / $g_x < 0$ / $g_x \geq 0$ and dx < 0	ahead / alongside / behind
across the road	$g_y < 0.3$ m (path margin) / otherwise by the sign of dy	in the ego's path / left (dy > 0) / right

Combining them gives **front, front-left, front-right, left, right, rear, rear-left, rear-right** (alongside + left = *left*, alongside + right = *right*). Alongside + in the path means the bodies overlap, i.e. contact; it is counted as *front*.

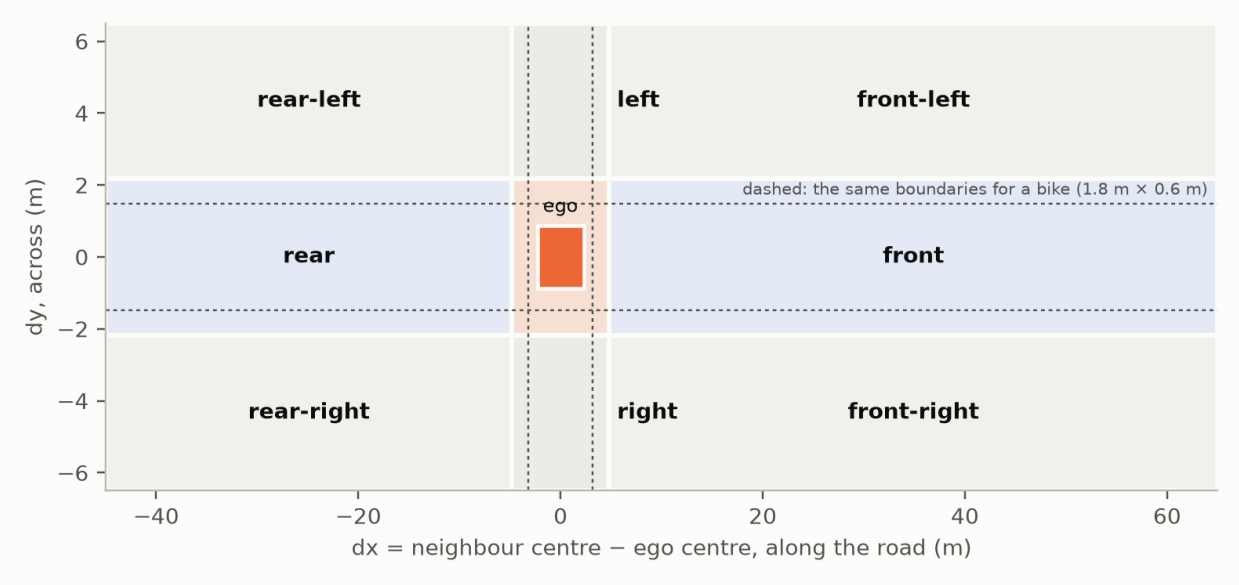


Figure 1. Sector boundaries in centre-offset coordinates for a car-sized neighbour (5 m × 2 m). The small central box (orange tint) is where the bodies overlap: contact, counted as front. Because the rules use box gaps, the boundaries move with the neighbour's size: dashed lines show them for a bike. Vehicles farther than 100 m (g_x) or 10 m (g_y) are ignored, and the controller only passes vehicles within 150 m (config `encoding.range_m`).

4. One vehicle per sector: the most critical

If several vehicles fall into the same sector, the sector's slot holds the **most critical** one: the highest inverse time-to-contact, and among equals the smallest box distance $\sqrt{(\max(g_x,0)^2 + \max(g_y,0)^2}$. The time-to-contact is the time until the two bodies (axis-aligned boxes) would first touch if both kept their current velocities; it is infinite if they never would.

The 9 values of a sector (empty sector: the defaults in the last column):

#	value	definition	empty
0	present	1 if a vehicle is encoded in this sector	0
1	gap_long	$\max(g_x, 0)$, capped at 100 m	100
2	gap_lat	$\max(g_y, 0)$, capped at 10 m	10
3	dvx	vehicle vx − ego vx (m/s); < 0 ahead = the ego is catching up	0
4	dvy	vehicle vy − ego vy (m/s)	0
5	inv_ttc	1 / time-to-contact (1/s); 0 = never touches, capped at 10 (also when already touching)	0
6	drac	braking needed to avoid contact along x: closing² / (2·max(gap_long, 0.1)); front: what the ego must brake, rear: what the follower must brake (m/s²)	0
7	length	vehicle length (m): bike 1.8, auto 2.6, car 5.0, bus/truck 12.0	0
8	width	vehicle width (m)	0

5. The 8 global values

#	value	definition
72	vx	ego forward speed (controller state, m/s)
73	vy	ego sideways speed (m/s)
74	heading	angle of the ego's velocity to the road, atan2(vy, vx) (rad)
75	y	ego lateral position (road centre = 0, edges at ±5.25 m)
76	clear_left	body clearance to the left road edge (m)
77	clear_right	body clearance to the right road edge (m)
78	density	number of vehicles whose centre is within ±50 m along the road (any lateral position)
79	mean_dvx_ahead	mean dvx of those vehicles that are ahead (m/s; negative = slower traffic ahead)

Index map. Sector k (0 = front, 1 = front-left, 2 = front-right, 3 = left, 4 = right, 5 = rear, 6 = rear-left, 7 = rear-right) occupies indices 9k … 9k+8, in the order of the table in section 4; the global values are 72–79. Column names in `encoding.csv` are `<sector>_<value>`, e.g. `front_left_inv_ttc`.

6. Worked example

A made-up scene, encoded with the real `encode()` function. The ego drives at 12 m/s and drifts left at 0.3 m/s. Arrows show each vehicle's velocity *relative to the ego*.

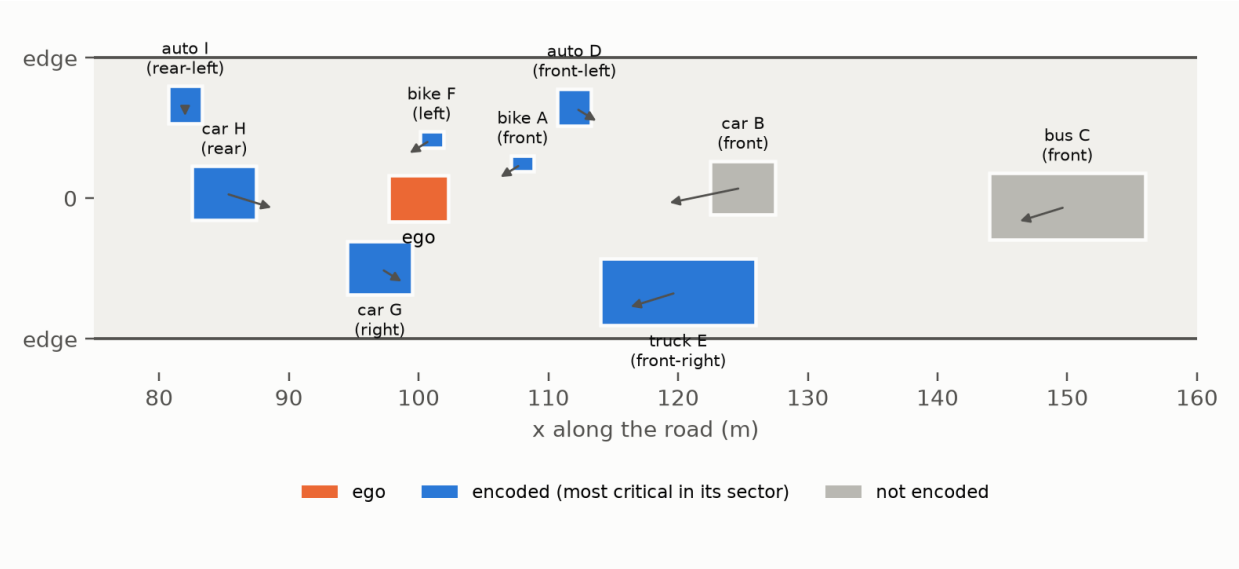


Figure 2. Which vehicle fills which sector. Blue vehicles are encoded; grey ones lose to a more critical vehicle in their sector.

sector	vehicle	gap_long	gap_lat	dvx	dvy	inv_ttc	drac	L × W
front	bike A	4.8	0.13	-1.0	-0.3	0.208	0.10	1.8 × 0.6
front-left	auto D	8.4	1.83	+1.0	-0.3	0.000	0.00	2.6 × 1.4
front-right	truck E	11.7	1.38	-2.0	-0.3	0.000	0.17	12 × 2.5
left	bike F	0.0	1.03	-1.0	-0.3	0.292	5.00	1.8 × 0.6
right	car G	0.0	0.73	+1.0	-0.3	0.000	5.00	5 × 2
rear	car H	10.2	0.00	+2.0	-0.3	0.196	0.20	5 × 2
rear-left	auto I	14.4	1.93	+0.0	-0.3	0.000	0.00	2.6 × 1.4
rear-right	*(empty)*	100	10	0	0	0	0	0 × 0

Global: vx = 12, vy = 0.3, heading = 0.025 rad, y = 0, clear_left = 4.38 m, clear_right = 4.38 m, density = 9, mean_dvx_ahead = -1.33 m/s.

What the example shows:

- **Front:** three vehicles are in the path. The bike A, only partly in the path and closing at 1 m/s, would touch in 4.8 s; the car B, closing at 3 m/s but farther away, in 6.7 s; the bus C never: the ego, drifting left at 0.3 m/s, leaves the bus's path before it gets there. The bike is the most critical, so it fills the slot, and the car and the bus are **not encoded**.
- **Left / right:** the bike F and the car G overlap the ego along the road, so they are alongside. Their drac reads 5 and 5 m/s², although neither is in front of or behind the ego: an artefact, see limitation 3 below.
- **Rear:** the car H closes from behind at 2 m/s; its drac is what H would have to brake.
- **Rear-right is empty**, so its slot holds the defaults (100, 10, then zeros).

7. Properties and limitations

What it guarantees: always 80 numbers; the same scene gives the same vector whatever the number or order of the vehicles; relative motion is included (dvx, dvy, inv_ttc, drac); lanes play no role; vehicle type enters through gaps and length/width.

Limitations (worth knowing before training on it):

- **One vehicle per sector.** Everything except the most critical vehicle in each sector is invisible (the car and the bus in the example). Density and mean_dvx_ahead only give a coarse summary.
- **Jumps.** An empty sector reads 100 / 10 / 0…; when a vehicle enters, the values jump. Changes of the most critical vehicle also make the slot jump.
- **drac is meaningless in the left / right sectors.** Those vehicles overlap the ego along the road (gap_long = 0), so the formula divides by its 0.1 m floor and produces large values whenever dvx ≠ 0.
- **No history.** Accelerations (a braking leader) and intent (a vehicle starting to cut in) are not visible beyond the current dvy.
- **Axis-aligned boxes.** Heading is ignored for the neighbours' bodies.
- **Not used for control yet** and only computed with `--log-encoding`.

Possible improvements (discussed earlier): closeness = exp(−gap/scale) instead of raw gaps (no jumps, empty = 0), a vehicle count per sector, drac only for front/rear, and left/right mirroring as data augmentation.

8. How to get it

`python dipf_controller.py --seed 1000 --log-encoding` writes `results/results_1000/encoding.csv`: one row per control step, a `time` column plus the 80 features. In code: `encoding.encode(ego, vehicles, road_edges, ego_length, ego_width)` returns the list, `encoding.FEATURES` the names in the same order.